Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Semafor Inteligent cu Prioritizare a Vehiculelor de Urgență

Firican Alexia

Grupa 422E

București 2025

Cuprins

Introducere

Capitolul 1.Specificații tehnice, pagina 5

Capitolul 2.Circuit logic combinațional, paginile 6-7

Capitolul 3.Circuit logic secvențial, paginilie 8-11

Capitolul 4.Proces secvențial, paginile 12-16

Capitolul 5.Implementarea codului sursă, paginile 17-20

Capitolul 6.Testarea funcționalității sistemului, paginile 21-25

Concluziile proiectului, pagina 26

Lista figurilor și lista tabelelor

Listă figuri:

3.1 Graf CLS semafor clasic, pagina 8

3.2 Graf CLS semafor inteligent, pagina 8

4.1 Schema bloc a semaforului, pagina 12

4.2 Graf PS mod software, pagina 13

4.3 Graf PS mod software, pagina 13

4.4 Graf PS mod software, pagina 14

Listă tabele:

2.1 Rezolvarea calculului unei singure măști, pagina 6

2.2 Tabela de adevăr a CLC-ului, pagina 6

3.1 Tabela de stări, pagina 9

3.2 Tabel de semnale relevante pentru CLS, pagina 9

3.3 Calculul măștii CLS, pagina 10

Introducere

Am decis să „construiec” un semafor inteligent cu prioritizarea vehiculelor de urgență deoarece la noi în țară nu am văzut să se utilizeze astfel de semafoare și mi s-ar părea folositoare, mai ales în orașele cu populație mare.

Proiectul include, pe lângă funcțiile normale ale unui semafor (tranzițiile verde-galben-roșu), încă două funcții, care-i dau rolul de semafor inteligent: modul urgență și prelungirea culorii verde.

Deși în cerința problemei nu se menționează de utilizarea unor butoane, ci numai că tranzițiile verde-galben-roșu se fac printr-un cronometru intern, am decis ca pentru testarea debugger-ului să adaug două butoane pentru stările speciale ale semaforului. Pentru cele două stări suplimentare, dacă apăs un buton, să se aprindă un led care să demonstreze că semaforul funcționează și că preia semnalul pentru starea respectivă.

Pentru realizarea logicii proiectului, am folosit:

1)Un circuit logic combinațional (CLC) care gestionează fluxul de trafic, adică senzorii specifici stării traficului.

2)Un circuitul logic secvențial (CLS) care m-a ajutat la determinarea stărilor semaforului, acestea fiind 6 la număr.

3)Procesul secvențial (PS), cel mai complex, bazat pe funcția switch-case, iar variabila tick, cu rol de temporizator, gestionează timpul de tranziții dintre stări: după 500 de tick-uri, aproximativ 10s semaforul se schimbă din verde în galben; după 150 de tick-uri, aproximativ 3s, se schimbă în roșu; după iar 500 de tick-uri, se schimbă în verde; timpul de prelungire verde l-am ales cam 25s, adică după 750 tick-uri. La fiecare tranziție, am resetat tick-ul ca semaforul să facă corect tranzițiile și ca acesta să rămână suficient timp în fiecare stare.

Pentru a pune laolaltă CLC-ul, CLS-ul și PS-ul, am scris un cod C în aplicația CODEVisionAVR Evaluation unde am ales microcontroler-ul Atmega164p și am considerat registrul PORTB pentru ieșiri și registrul PIND pentru intrări, programul rulând cu o frecvență de aproximativ 20ms.

Capitolul 1.Specificațiile proiectării

Intrări

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
		URG	PREL	SET	SR	SI	SP

Ieșiri

LED7	LED6	LED5	LED4	LED3	LED2	LED1	LED0
URG	PREL	V	G	R		Flux_trafic	

SP-senzori de presiune

SI-senzori infraroșu

SR-semnal radio

URG-declanșare mod urgență

PREL-prelungire culoare verde

SET-semnalul de tranziție automată între stările semaforului (tick)

Led-urile funcționează pe logica inversă, adică sunt aprinse în 0.

URG-led urgență

PREL-led prelungire

V-led verde

G-led galben

R-led roșu

Registrul PORTB-ieșiri

Registrul PIND-intrări

Switch-urile o să fie active în 1.

Frecvența de ceas este aproximativ 20ms.

Capitolul 2.Circuit logic combinațional

Folosesc CLC pentru a folosi senzorii semaforului în detecția fluxului de trafic, alături de un semnal pentru a prioritiza vehiculele de urgență, toate fiind active în 1.

Pentru că utilizez 3 senzori, am 3 biți pe intrare= 8 combinații de intrare

LED1 și LED0, deoarece sunt 2 biți= 4 combinații pe ieșire, arată cum este traficul:

11-fără mașini, rămâne culoarea roșu

01-flux mare de trafic, se prelungește verdele

10- flux normal de trafic

11-mod urgență, verde instantaneu

Rezolvarea cu calculul unei singure măști Tabel 2.1

INTRARE			URG	PREL		SR	SI	SP	&
MASCA	0	0	0	0	0	1	1	1	=0x07
INDEX	0	0	0	0	0	SR	SI	SP	

Tabela de adevăr a CLC-ului

INTRARE				IEȘIRE			
SR	SI	SP	Semnificație	LED1	LED0	Semnificație	Val Hexa
0	0	0	-	1	1	Șosea liberă	0x03
0	0	1	Activ SP	1	0	Trafic normal	0x02
0	1	0	Activ SI	1	0	Trafic normal	0x02
0	1	1	Activ SP,SI	0	1	Trafic intens	0x01
1	0	0	Activ SR	0	0	Mod urgență	0x00
1	0	1	Activ SR,SP	0	0	Mod urgență	0x00
1	1	0	Activ SR,SI	0	0	Mod urgență	0x00
1	1	1	Activ SR,SI,SP	0	0	Mod urgență	0x00

Tabel 2.2

LED1: !SR

LED0: !(SI OR SP)

*Când SI și SP sunt active, am considerat că pragul critic este depășit.

Tabela de adevăr a CLC-ului va fi definită ca vector TAB cu 8 elemente:

Char TAB[8] = {0x03,0x02,0x02,0x01,0x00, 0x00, 0x00, 0x00}

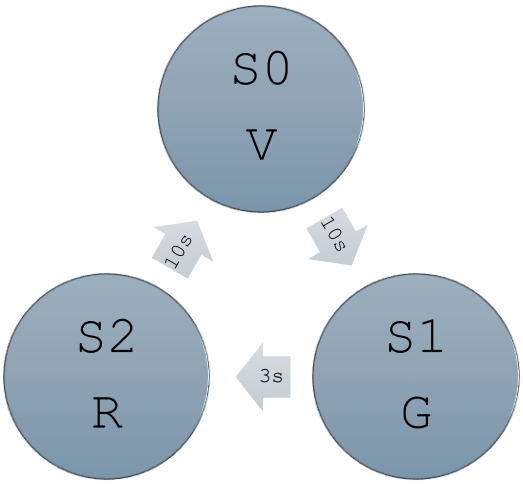
CLC poate fi implementat ca o funcție ce poate fi apelată în interiorul codului la momentele precizate:

```
{
 char in;
 char flux_trafic;
 char TAB[] = {0x03,0x02,0x02,0x01,0x00, 0x00, 0x00, 0x00};
 void CLC(void) {
     char selectie;
     selectie = in & 0x07;
     flux_trafic = TAB[selectie];
}
```

Capitolul 3.Circuit logic secvențial

Dacă semafoul funcționează cu cronometrul intern, fără a adăuga cele 2 stări(prelungirea timpului de verde sau schimbarea pe verde instantaneu), atunci graful ar arăta așa:

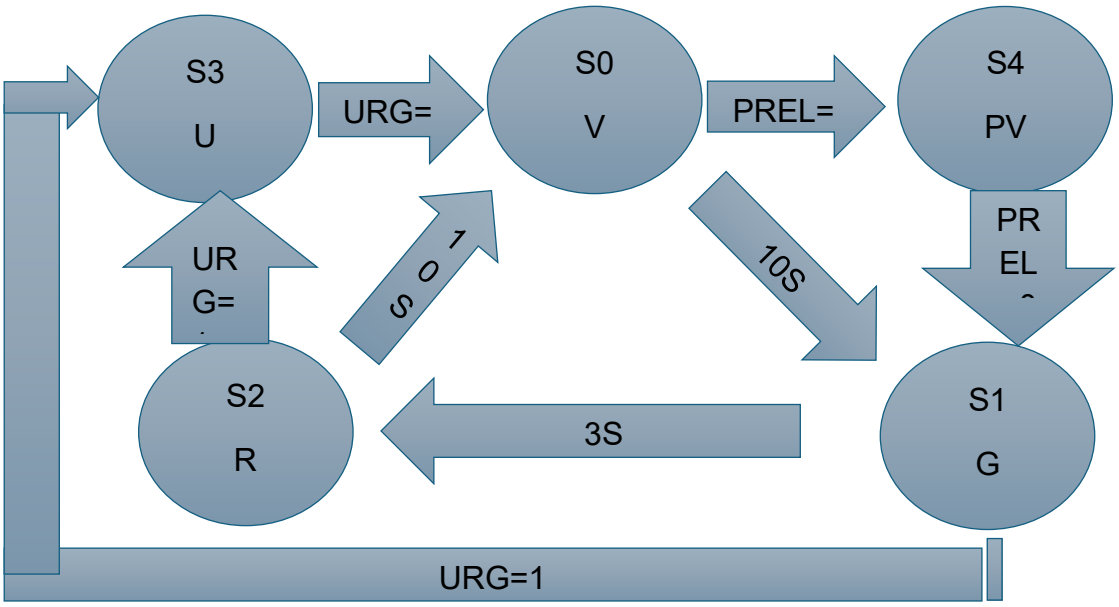
Graf CLS semafor clasic



Graf 3.1

Adăugând și celelalte 2 stări cu 2 butoane URG și PREL:

Graf CLS semafor inteligent



*PV-prelungire verde, U-urgență

Graf 3.2

Tabel stări

Stare	Semnificație	Valoare ieșire	
		Binar	Zecimal
0	Stare inițială VERDE(10s)	000	0
1	GALBEN(3s)	001	1
2	ROȘU(10s)	010	2
3	URGENȚĂ	011	3
4	PRELUNGIRE VERDE	100	4
5	GALBEN după urgență	101	5

Tabel 3.1

Tabel de semnale relevante pentru CLS și determinarea măștilor necesare

Stare	Adresa tabelei de semnale relevante
0	A0
1	A1
2	A2
3	A3
4	A4
5	A5

Tabel 3.2

SET->SW3->0x08

PREL-> SW4->0x10

URG-> SW5->0x20

Tabelul de semnale relevante

Acest tabel are primele 3 stări mai lungi deoarece m-am gândit în funcție de al doilea graf să fac tranzițiile de stări.

```
char A0[] = { 0x08, 1, 0x10, 4, 0x20, 3, T, 0 };

char A1[] = { 0x08, 2, 0x20, 3, T, 1 };

char A2[] = { 0x08, 0, 0x20, 3, T, 2 };

char A3[] = { 0x08, 5, T, 3 };

char A4[] = { 0x08, 1, T, 4 };

char A5[] = { 0x08, 2, T, 5 };
```

Calculul măștii CLS:

Intrare			URG	PREL	SET	SR	SI	SP
Masca_SET	0	0	1	1	1	0	0	0

=0x38

Tabel 3.3

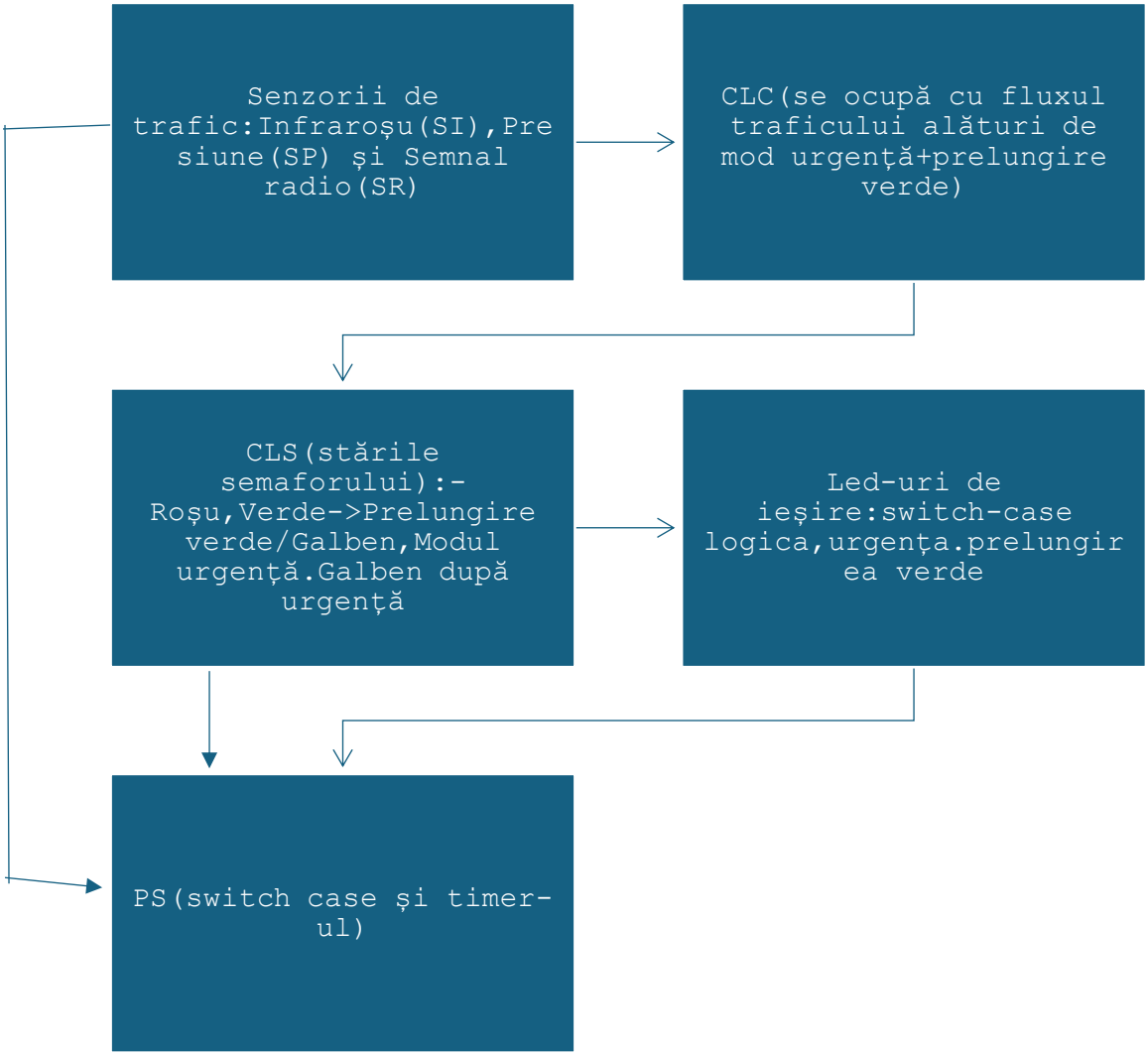
Cod CLS

```
#define T  0xFF
#define NR 6

char Q;
char in;
char out;
// Tabela de adrese a tabelelor de semnale
char *TABA[NR] = {A0, A1, A2, A3, A4, A5};
// Tabelele de semnale relevante
char A0[] = {0x08, 1, 0x10, 4, 0x20, 3, T, 0};
char A1[] = {0x08, 2, 0x20, 3, T, 1};
char A2[] = {0x08, 0, 0x20, 3, T, 2};
char A3[] = {0x08, 5, T, 3};
char A4[] = {0x08, 1, T, 4};
char A5[] = {0x08, 2, T, 5};
void cls(void) {
  char i;
  char *adr;
  char ready;

  adr = TABA[Q];
  i = 0;
  ready = 0;
```

```
  while (!ready) {
    if ((in & 0x38) == *(adr + i))
     {  // 0x38 = SET (bit3), PREL (bit4), URG (bit5)
      Q = *(adr + i + 1);
      ready = 1;
    }
    else if (*(adr + i) == T)
 {
      ready = 1;
    }
    else {
      i = i + 2;
    }
  }
  out = Tout[Q];
}
```

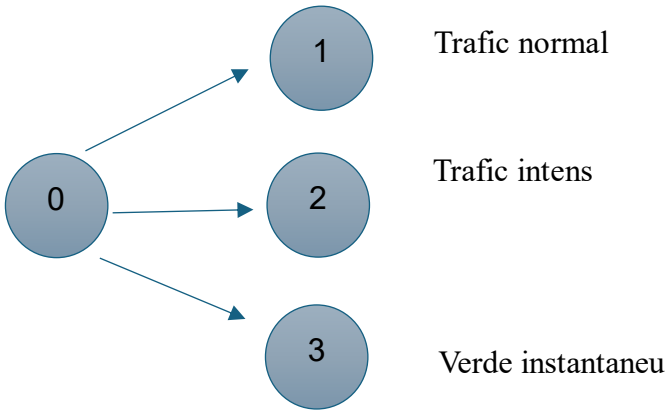
Capitolul 4.Proces secvențial

Schema bloc a semaforului(4.1)



Funcționarea modului software al semaforului inteligent se bazează pe un proces secvențial. Ca să creez graful, am considerat că semaforul este funcționabil în starea 0, care se ramifică în alte 3 tranziții: starea normală, pentru un flux normal al traficului (V-G-R), modul cu timp ajustabil (PV în funcție de trafic) și prioritatea vehiculelor de urgență (U, comutarea forțată pe verde).

Graf PS mod software

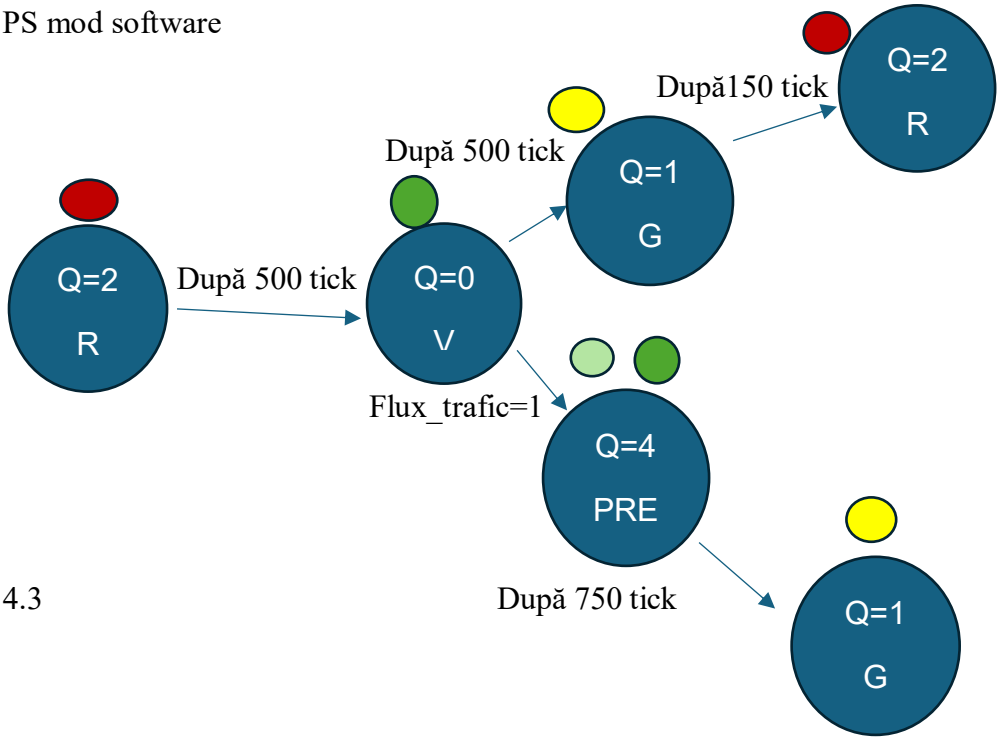


Graf 4.2

Când pornesc semaforul, prima stare a acestuia este din roșu, adică din starea a 2 a, iar tranzițiile se fac prin temporizatorul pe care l-am numit tick în cod.

Nu există o stare de reset a semaforului, resetul făcându-se practic printr-o reîncărcare.

Graf PS mod software



Graf 4.3

1)Se aprinde led-ul ROȘU

2)Se aprinde led-ul VERDE, se stinge cel ROȘU

3)Se aprinde led-ul GALBEN, se stinge cel VERDE

4)Se aprinde led-ul ROȘU, se stinge cel GALBEN

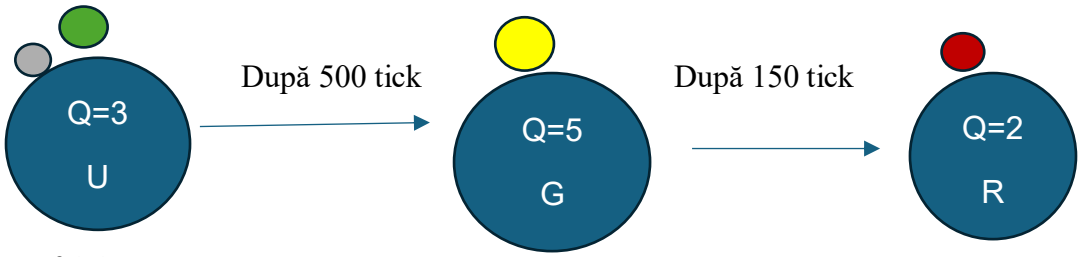
Până aici a fost pe ramura de sus aprinderea led-urilor pentru funcționarea clasică a semaforului

5)Rămâne aprins led-ul VERDE și se aprinde led-ul care semnalizează PRELUNGIREA VERDE

6)Se aprinde led-ul GALBEN, se sting led-urile VERDE și PRELUNGIREA VERDE

Oricând activez butonul URG:

Graf PS mod software



Graf 4.4

1)Se aprind led-urile URG și V

2)Se aprinde numai led-ul GALBEN, se sting URG și V

3)Se aprinde led-ul ROȘU, se stinge GALBEN

*Am pus cerculețe colorate deasupra fiecări stări în parte pentru a reprezenta ce led-uri se aprind.

Switch-case este structura cea mai potrivită pentru procesul secvențial cu stări finite pentru că pot modifica cu ușurință codul ca să adaug sau să scot stări.

Fiecare caz din switch corespunde unei stări a semaforului, cu durata de 500 de tick-uri/10 secunde (din verde-galben sau roșu-verde), 150 de tick-uri/3 secunde (din galben în roșu), 750 tick-uri/15 secunde (din prelungire verde în galben). Contorul tick se resetează la finalul fiecărei perioade prestabilite pentru ca semaforul să funcționeze corect.

Codul în C ține cont de starea curentă în care se află semaforul. La fiecare case am pus un comentariu pentru stare.

Cod PS

```
if ((in & URG) == URG && Q != 3) {
    in = URG;
    cls();
    tick = 0;
} else {
    tick++;
    if (Q == 0 && tick >= 500 && flux_trafic == 1) {
        in = PREL;
        cls();
        tick = 0;
    } else {
        switch (Q) {
            case 0: // Verde
                if (tick >= 500) {
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
            case 1: // Galben
                if (tick >= 150) {
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
            case 2: // Roșu
                if (tick >= 500) {
```

```
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
            case 3: // Urgență
                if (tick >= 500) {
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
            case 4: // Prelungire verde
                if (tick >= 750) {
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
            case 5: // Galben după urgență
                if (tick >= 150) {
                    in = SET;
                    cls();
                    tick = 0;
                }
                break;
        }
    }
}
```

Capitolul 4.Implementarea codului sursă

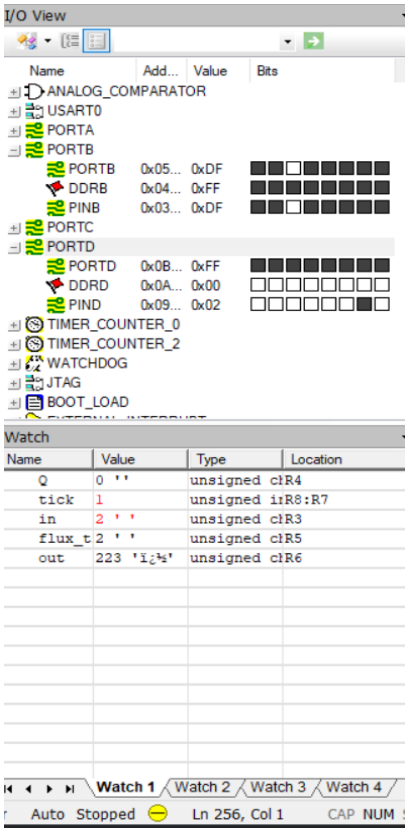
```
#include <mega164.h>

#define SET   0x08

#define PREL  0x10

#define URG   0x20

#define T     0xFF

#define NR    6

// Declare your global variables here

char Q = 2;

char in = 0;

char out = 0;

char flux_trafic = 0;

unsigned int tick = 0;


char TAB[8] = {0x03, 0x02, 0x02, 0x01, 0x00,
0x00, 0x00, 0x00};


char A0[] = {0x08, 1, 0x10, 4, 0x20, 3, T, 0};

char A1[] = {0x08, 2, 0x20, 3, T, 1};

char A2[] = {0x08, 0, 0x20, 3, T, 2};

char A3[] = {0x08, 5, T, 3};

char A4[] = {0x08, 1, T, 4};

char A5[] = {0x08, 2, T, 5};

char* TABA[NR] = {A0, A1, A2, A3, A4, A5};

 char Tout[NR] = {

    0xDF,

    0xEF,

    0xF7,

    0x5F,

    0x9F,

    0xEF

};

// Timer 0 overflow interrupt service routine

interrupt [TIM0_OVF] void timer0_ovf_isr(void)

{

// Reinitialize Timer 0 value

TCNT0=0x3C;

// Place your code here

}

// Timer 0 output compare B interrupt service
routine

interrupt [TIM0_COMPB] void
timer0_compb_isr(void)

{

// Place your code here

}

void clc(void) {

    char selectie = in & 0x07;

    flux_trafic = TAB[selectie];

}

void cls(void) {

    char* adr = TABA[Q];

    char i = 0;

    char ready = 0;

    while (ready == 0) {

        if ((in & 0x38) == *(adr + i)) {

            Q = *(adr + i + 1);

            ready = 1;

        } else if (*(adr + i) == T) {

            ready = 1;

        } else {

            i = i + 2;

        }

    }

    out = Tout[Q];

}

void main(void)

{

// Declare your local variables here

 DDRD = 0x00;

    PORTD = 0xFF;

    DDRB = 0x3F;
```

```
    PORTB = 0x00;



    out = Tout[Q];

    PORTB = out;

// Clock Oscillator division factor: 1

#pragma optsize-

CLKPR=(1<<CLKPCE);

CLKPR=(0<<CLKPCE) | (0<<CLKPS3) | (0<<CLKPS2)
| (0<<CLKPS1) | (0<<CLKPS0);

#ifdef _OPTIMIZE_SIZE_

#pragma optsize+

#endif

// Input/Output Ports initialization

// Port A initialization

// Function: Bit7=In Bit6=In Bit5=In Bit4=In
Bit3=In Bit2=In Bit1=In Bit0=In

DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) |
(0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1)
| (0<<DDA0);

// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T
Bit2=T Bit1=T Bit0=T

PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5)
| (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) |
(0<<PORTA1) | (0<<PORTA0);

// Port B initialization

// Function: Bit7=Out Bit6=Out Bit5=Out
Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out

DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) |
(1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1)
| (1<<DDB0);

// State: Bit7=1 Bit6=1 Bit5=1 Bit4=1 Bit3=1
Bit2=1 Bit1=1 Bit0=1

PORTB=(1<<PORTB7) | (1<<PORTB6) | (1<<PORTB5)
| (1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) |
(1<<PORTB1) | (1<<PORTB0);

// Port C initialization

// Function: Bit7=In Bit6=In Bit5=In Bit4=In
Bit3=In Bit2=In Bit1=In Bit0=In

DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) |
(0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1)
| (0<<DDC0);

// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T
Bit2=T Bit1=T Bit0=T

PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5)
| (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) |
(0<<PORTC1) | (0<<PORTC0);

// Port D initialization

// Function: Bit7=In Bit6=In Bit5=In Bit4=In
Bit3=In Bit2=In Bit1=In Bit0=In

DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) |
(0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1)
| (0<<DDD0);

// State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P
Bit2=P Bit1=P Bit0=P

PORTD=(1<<PORTD7) | (1<<PORTD6) | (1<<PORTD5)
| (1<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) |
(1<<PORTD1) | (1<<PORTD0);

// Timer/Counter 0 initialization

// Clock source: System Clock

// Clock value: 9.766 kHz

// Mode: Normal top=0xFF

// OC0A output: Disconnected

// OC0B output: Disconnected

// Timer Period: 20.07 ms

TCCR0A=(0<<COM0A1) | (0<<COM0A0) | (0<<COM0B1)
| (0<<COM0B0) | (0<<WGM01) | (0<<WGM00);

TCCR0B=(0<<WGM02) | (1<<CS02) | (0<<CS01) |
(1<<CS00);

TCNT0=0x3C;

OCR0A=0x00;

OCR0B=0x00;

// Timer/Counter 1 initialization

// Clock source: System Clock

// Clock value: Timer1 Stopped

// Mode: Normal top=0xFFFF

// OC1A output: Disconnected

// OC1B output: Disconnected

// Noise Canceler: Off

// Input Capture on Falling Edge

// Timer1 Overflow Interrupt: Off

// Input Capture Interrupt: Off

// Compare A Match Interrupt: Off

// Compare B Match Interrupt: Off

TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1)
| (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);

TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) |
(0<<WGM12) | (0<<CS12) | (0<<CS11) |
(0<<CS10);
```

```
TCNT1H=0x00;

TCNT1L=0x00;

ICR1H=0x00;

ICR1L=0x00;

OCR1AH=0x00;

OCR1AL=0x00;

OCR1BH=0x00;

OCR1BL=0x00;

// Timer/Counter 2 initialization

// Clock source: System Clock

// Clock value: Timer2 Stopped

// Mode: Normal top=0xFF

// OC2A output: Disconnected

// OC2B output: Disconnected

ASSR=(0<<EXCLK) | (0<<AS2);

TCCR2A=(0<<COM2A1) | (0<<COM2A0) | (0<<COM2B1)
| (0<<COM2B0) | (0<<WGM21) | (0<<WGM20);

TCCR2B=(0<<WGM22) | (0<<CS22) | (0<<CS21) |
(0<<CS20);

TCNT2=0x00;

OCR2A=0x00;

OCR2B=0x00;

// Timer/Counter 0 Interrupt(s) initialization

TIMSK0=(1<<OCIE0B) | (0<<OCIE0A) | (1<<TOIE0);

// Timer/Counter 1 Interrupt(s) initialization

TIMSK1=(0<<ICIE1) | (0<<OCIE1B) | (0<<OCIE1A)
| (0<<TOIE1);

// Timer/Counter 2 Interrupt(s) initialization

TIMSK2=(0<<OCIE2B) | (0<<OCIE2A) | (0<<TOIE2);

// External Interrupt(s) initialization

// INT0: Off

// INT1: Off

// INT2: Off

// Interrupt on any change on pins PCINT0-7:
Off

// Interrupt on any change on pins PCINT8-15:
Off

// Interrupt on any change on pins PCINT16-23:
Off

// Interrupt on any change on pins PCINT24-31:
Off

EICRA=(0<<ISC21) | (0<<ISC20) | (0<<ISC11) |
(0<<ISC10) | (0<<ISC01) | (0<<ISC00);

EIMSK=(0<<INT2) | (0<<INT1) | (0<<INT0);

PCICR=(0<<PCIE3) | (0<<PCIE2) | (0<<PCIE1) |
(0<<PCIE0);

// USART0 initialization

// USART0 disabled

UCSR0B=(0<<RXCIE0) | (0<<TXCIE0) | (0<<UDRIE0)
| (0<<RXEN0) | (0<<TXEN0) | (0<<UCSZ02) |
(0<<RXB80) | (0<<TXB80);

// USART1 initialization

// USART1 disabled

UCSR1B=(0<<RXCIE1) | (0<<TXCIE1) | (0<<UDRIE1)
| (0<<RXEN1) | (0<<TXEN1) | (0<<UCSZ12) |
(0<<RXB81) | (0<<TXB81);

// Analog Comparator initialization

// Analog Comparator: Off

// The Analog Comparator's positive input is

// connected to the AIN0 pin

// The Analog Comparator's negative input is

// connected to the AIN1 pin

ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) |
(0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1)
| (0<<ACIS0);

ADCSRB=(0<<ACME);

// Digital input buffer on AIN0: On

// Digital input buffer on AIN1: On

DIDR1=(0<<AIN0D) | (0<<AIN1D);

// ADC initialization

// ADC disabled

ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) |
(0<<ADIF) | (0<<ADIE) | (0<<ADPS2) |
(0<<ADPS1) | (0<<ADPS0);

// SPI initialization

// SPI disabled

SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) |
(0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1)
| (0<<SPR0);

// TWI initialization

// TWI disabled
```

```
TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) |
(0<<TWEN) | (0<<TWIE);

// Globally enable interrupts

#asm(„sei”)


while (1)
      {
      // Place your code here
 in = PIND;
        clc();


        if ((in & URG) == URG && Q != 3) {
            in = URG;
            cls();
            tick = 0;
        } else {
            tick++;


            if (Q == 0 && tick >= 500 &&
flux_trafic == 1) {
                in = PREL;
                cls();
                tick = 0;
            } else {
                switch (Q) {
                    case 0:
                        if (tick >= 500) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                    case 1:
                        if (tick >= 150) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                    case 2:
                        if (tick >= 500) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                    case 3:
                        if (tick >= 500) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                    case 4:
                        if (tick >= 750) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                    case 5:
                        if (tick >= 150) {
                            in = SET;
                            cls();
                            tick = 0;
                        }
                        break;
                }
            }
        }
        PORTB = out;
    }
```

Capitolul 5.Testarea funcționalității sistemului



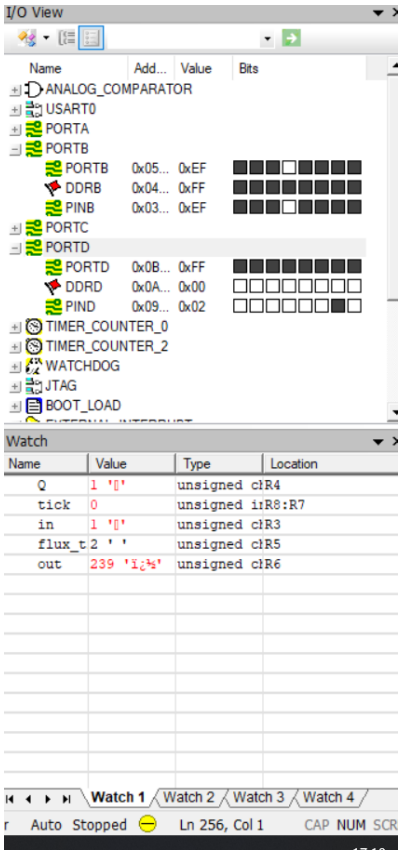
Trafic normal:culoarea verde

Pentru a alege fluxul normal de trafic, am apăsat pe PIND1 care corespunde senzorului SI pe care în tabelul CLC-ului l-am pus ca determinant al traficului normal(flux_trafic=2,10 în binar din tabelul meu). Q=0 este starea VERDE din CLS vizibilă în graf.

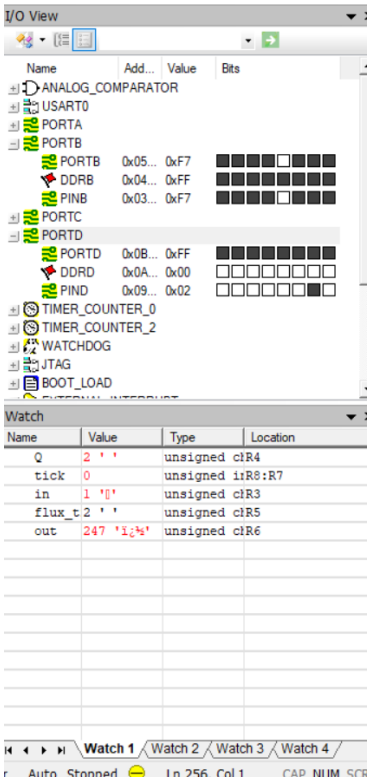
Intrarea este 2, ceea ce confirmă flux_trafic = 2 (char TAB[2]) din tabelul CLC.

Led-ul corespunzător culorii verde este LED5, care este corespunzător celui aprins.(out=223 sau 0xDF în hexa)

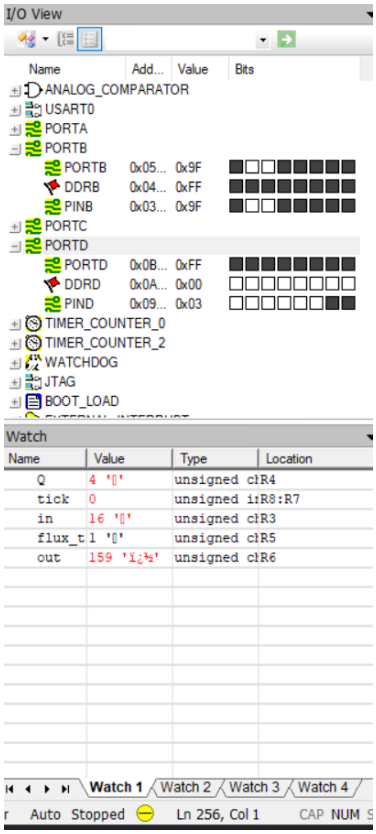
După ce tick, care este contorul meu de timp, trece de 500, atunci se va aprinde culoarea GALBEN tot în flux_trafic=2.



S-a intrat în starea 1, adică semaforul s-a făcut GALBEN. Q=1, char TAB[2] pentru că mă aflu tot în trafic normal, iar out=239 sau 0xEF, poziție corespunzătoare led-ului GALBEN.



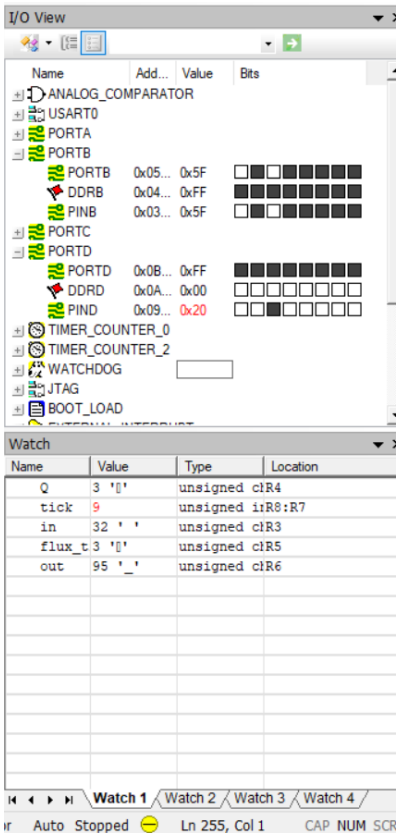
După 150 de tick-uri, semaforul s-a făcut ROȘU, Q=2,out=247 sau 0xF7 pentru LED3



Ca să activez Q=4 care este starea de PRELUNGIRE, am apăsat simultan SW0 și SW1 deoarece acestea declanșează aglomerația, iar în codul C butonul de PREL este activat când decide programul, nu manual de mine în debugger. După ce tick depășește valoarea 500, apare modul PRELUNGIRE VERDE in=16, adică 0x10, adică SW4 e activ, fiind PREL.

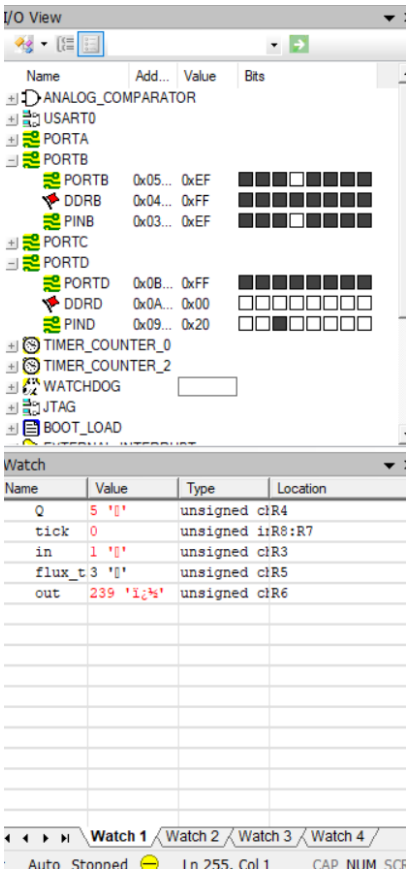
Flux_trafic = 1, este un flux de trafic intens, de aceea s-a activat PRELUNGIRE VERDE

Out=159 sau 0x9F ca să se aprindă LED5 și LED6

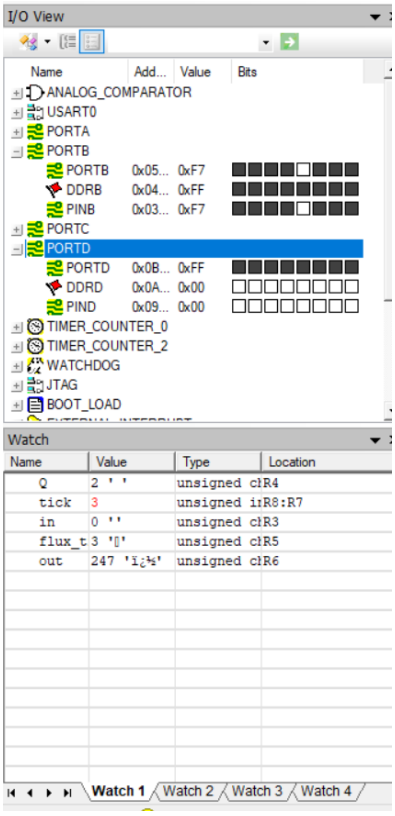


Activarea mod URGENȚĂ. Aici am utilizat manual butonul URG, iar astfel s-au aprins led-urile 7 și 5, specifice pentru a arăta explicit că este mod URG și apare verdele instantaneu.

Q=3 arată starea de URGENȚĂ, out=95 sau 0X5F pentru cele 2 led-uri aprinse

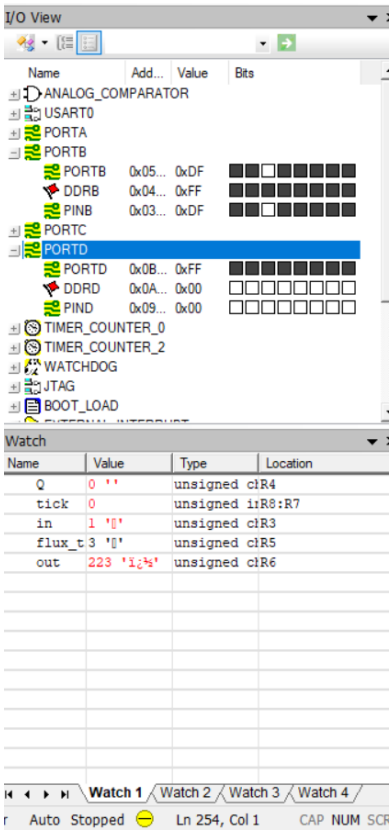


După 500 tick, trec în starea 5, GALBEN după ce a trecut mașina de urgență.



Debugger-ul inițial începe din starea 2, adică Q=2, unde ROȘU este aprins, LED3. Out=0xF7 sau 247, adică este aprins LED3.

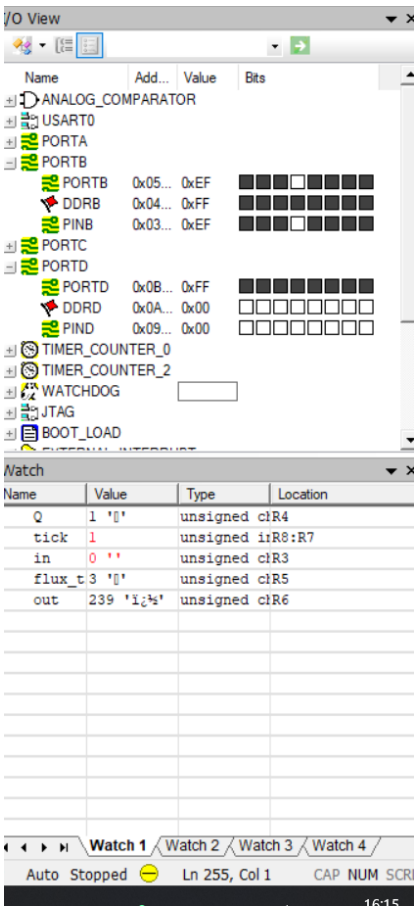
După 500, trece în verde.



Starea 0, Q=0, corespunzătoare culorii VERDE

Out=223 sau 0xDF, arătând ca LED5 este aprins

După 500 tick-uri, trece în galben.



Starea Q=1 este pentru GALBEN, OUT=239 sau 0xEF pentru LED4, flux_trafic=3

Concluzii

Proiectul acesta demonstrează aplicarea practică a conceptelor fundamentale din domeniul circuitelor logice, prin integrarea acestora în mod software.Astfel, am reușit să determin stările semaforului inteligent și totodată să le și testez. Condițiile suplimentare au fost mai dificil de realizat, dar am reușit să le pun în debugger și să le fac să funcționeze după cerințele mele, precum adăugarea unor led-uri care să semnaleze că am intrat în starea respectivă, dar și prin improvizarea unor butoane. Implementarea s-a bazat pe folosirea cronometrului intern, prin variabila tick. Prelungirea culorii verde a fost făcută pe baza semnalului de la cei doi senzori infraroșu și de presiune, iar activarea modului urgență am implementat-o pe butonul URG.

Folosul proiectului este de a îmbunătăți infracstructura urbană și de a scoate în evidență importanța microcontrolerelor în scopuri de automatizare.

De asemenea, semaforul inteligent cu prioritizare a vehiculelor de urgență poate fi implementat pe plăci de dezvoltare reale și poate fi un proiect în plină dezvoltare, cu și mai mulți senzori suplimentari adăugați.